



Drink-Sight, LLC 전략 발표 [뽀뽀 뽀뽀 TEAM]

▼ 목차

문제 상황 파악

데이터 EDA 및 전처리

1. 데이터 기본 EDA 및 전처리
2. 데이터 병합
3. 매출총이익(gross profit), 이익률(gross margin rate) 계산

brand_id 군집화를 통한 특징 파악

1. brand_id를 기준으로 한 이유
2. brand_id 군집화
 - 2-1. DBSCAN
 - 2-2. K-means
 - 2-3. 군집화 구조 정리
3. brand_id별 군집화 결과 해석
 - 3-1. [군집 0] 일반적인 10770개 상품 관리 전략 → ABC 관리법
ABC 관리법(ABC inventory planning)
 - 3-2. [군집 1] 초고가 제품 11개 상품 관리 전략 → 선주문 후발주 방식
선주문 후발주 방식
 - 3-3. [군집 2] 변동성 높은 402개 상품 → 회귀분석

군집 2 회귀 분석 및 기대효과

1. 회귀분석 목적
2. 회귀분석 모델 설명
3. 회귀 분석 결과 및 해석
 - 3-1. 회귀 분석 구조 정리
 - 3-2. 회귀 분석 결과
 - 3-3. 회귀 분석 결과 요약 및 해석
4. 회귀 분석 결과에 따른 군집 2 전략 제시
 - 4-1. 군집 2의 특징
 - 4-2. 전략 기대효과 검증
 - 4-3. 기대효과 시뮬레이션 결과

결론: brand_id별 재고 관리 가이드라인

0 0번 군집에 해당하는 일반적인 10777개 상품 관리 전략

1 1번 군집에 해당하는 초고가 11개 상품 관리 전략

2 2번 군집에 해당하는 변동성 높은 402개 상품 전략

문제 상황 파악

우리 Drink-sight는

여러 vendor에게서 다양한 주류 상품을 매입원가(purchase_price)에 발주(purchase)하여,

약 80여개의 소매점에서 판매가격(sales_price)에 판매(sale)함으로써 그 사이의 margin으로부터 이익을 얻는 주류 전문 소매 체인입니다.

저희 뽀빠뽀뽀 데이터 분석팀은 주류 상품별 특징에 주목하여,

약 80여개의 소매점에 통합적으로 제공할 **재고 관리 가이드라인**을 준비했습니다.

매출 총이익 극대화를 위해 다음과 같이

| **brand_id별 재고 관리 가이드라인**을 제시합니다.

1. **큰 특징 없는 일반적인 제품**: **ABC 관리법**을 활용하여 재고를 효과적으로 관리하는 전략이 필요합니다.
2. **고가 제품**: **초고가**이므로 정밀관리가 필요해 재고 관리비용이 많이 소요됩니다. 따라서 **최대한 재고를 적게 보유하는 전략**이 필요합니다.
3. **판매량과 재고량의 변동성이 높은 제품**: 회귀 분석을 통해 **일별 제품 수요량을 예측**해서 **효율적인 재고 관리**를 하는 전략이 필요합니다.

데이터 EDA 및 전처리

▼ 1. 데이터 기본 EDA 및 전처리

▼ 1-1. `begin_inventory`, `end_inventory`

- 기초 대비 기말 재고 확인

- 인벤토리 다양성: 8.7% 증가
- 총 재고량: 15.8% 증가
- 브랜드 다양성: 19.3% 증가
- 총 재고자산: 17.1% 증가



총 재고량과 총 재고자산이라는 두 가지 관점 모두에서 확인해보았을 때,

기초 대비 기말 재고가 증가한 것이 자명합니다.

재고가 증가한 상태가 지속되면, **재고 관리 비용이 증가**하여 장기적으로 **이익 극대화**에 방해가 될 수 있습니다. 따라서 **적절한 수준의 재고량을 유지하며 관리를 할 필요성**이 있습니다.

기초 대비 기말 재고가 증가한 것과 동시에, 인벤토리와 상품(브랜드) 종류도 증가했음을 알 수 있었습니다. 이때 **brand_id가 19.3% 증가**한 것에 주목할 만합니다.

따라서 저희 분석팀은 기초 대비 기말 재고의 증가와 **brand_id의 증가**는 큰 연관성이 있을 것이라고 보았고, **재고 과잉에 영향을 준 특정 브랜드(상품)**별로 그 특징을 파악하고 관리할 필요성에 주목하였습니다.

▼ `city` 결측값 처리

- `end_inventory`의 `city` 칼럼의 결측치 대체

```
store_id == 46 ⇒ city == "Canary"
```

▼ 1-2. sales

▼ 날짜 관련 파생변수 칼럼 추가

- sales_date ⇒ 월(month), 요일(dayofweek) 파생

▼ city 결측치 처리

- sales 의 city 칼럼의 결측치 대체

```
store_id == 46 ⇒ city == "Canary"  
store_id == 81 ⇒ city == "Spoonbill"
```

▼ **size** 칼럼 전처리

- 해당 상품의 총 용량(mL)을 나타내는 **volume** 칼럼 추가
 - 단위 통일

- 1 oz $\Rightarrow$ 29.5735 mL
- 1 gallon $\Rightarrow$ 3785.41 mL
- 1 L $\Rightarrow$ 1000mL

- 패키지 처리

- 250mL 4 Pk $\Rightarrow$ 1000mL

- 증정품 처리

- item에 VAP 표기가 있는 경우 $\Rightarrow$ 50mL 증정품 추가된 것으로 처리

- 예외 처리

- 25, 25.0 등 $\Rightarrow$ 25mL 샷 15개 묶음으로 처리 $\Rightarrow$ 375mL

▼ 1-3. purchases

▼ city 결측치 처리

- sales의 city 칼럼의 결측치 대체

```
store_id == 46 ⇒ city == "Canary"
store_id == 81 ⇒ city == "Spoonbill"
```

▼ size 결측치 처리 (3개)

- size의 NaN 값을 동일 item의 다른 구매내역을 이용하여 대체

- Pinnacle Rainbow Sherbet ⇒ 750ml로 대체
- Skinnygirl Pina Colada ⇒ 750ml로 대체
- Alabaster 07 Tinta de Toro ⇒ 750ml로 대체
 - purchases 데이터에서는 다른 구매내역이 없지만 sales 데이터와 inventory 데이터에서 찾아보니 모두 750ml이므로 750ml로 결측치를 대체

▼ 1-4. purchases_prices

- price 결측치 처리 (2개)
 - brand_id == 4202 인 행

- brand_id == 4202 인 행의 vendor인 'BACARDI USA INC'를 purchases 데이터에서 찾아본 결과, 이 vendor에서는 Bacardi Grapefruit Rum이라는 item 한 가지만 매입함 (brand_id == 4092)

따라서 brand_id == 4202 인 행은 잘못된 정보로 간주하고 [삭제하기로 결정](#)

- brand_id == 2166 인 행

- brand_id == 2166 인 행은 price가 0임.
sales 데이터에서 찾아보니 brand_id == 2166 의 price 는
 - 54.99 → 785번
 - 59.99 → 153번
 - 65.99 → 124번

으로 나타남.

따라서 brand_id == 2166 인 행의 price 는 [최빈값인 54.99로 대체하기로 결정](#)

▼ 1-5. **vendor_invoice**

▼ **approval** 결측치 처리 및 의미 확인

- NaN : 5169
- Frank Delahunt : 374

approval 칼럼은 고가의 상품에 대해 승인이 필요한 경우,
'Frank Delahunt'이 승인한 정보를 담은 것이라는 것을 확인함.

- **Frank Delahunt** → 비허가(0)
- **NaN** → 허가(1)

으로 대체

▼ 2. 데이터 병합

▼ 2-1. 재고 데이터: `inventory`

`begin_inventory` + `end_inventory`

▼ 병합된 `inventory` 데이터의 info

Int64Index: 257496 entries, 0 to 257495

Data columns (total 13 columns):

#	Column	Non-Null Count		Dtype
---	-----	-----		-----
0	inventory_id	257496	non-null	object
1	store_id	257496	non-null	int64
2	city	257496	non-null	object
3	brand_id	257496	non-null	int64
4	item	257496	non-null	object
5	size	257496	non-null	object
6	on_hand_begin	206529	non-null	float64
7	sales_price_begin	206529	non-null	float64
8	begin_date	206529	non-null	object
9	on_hand_end	224489	non-null	float64
10	sales_price_end	224489	non-null	float64
11	end_date	224489	non-null	object
12	on_hand_diff	173522	non-null	float64

dtypes: float64(5), int64(2), object(6)

▼ 2-2. 공급 데이터: `purchase_vendor`

`purchases` + `vendor_invoice`

- 부피(`volume`) 기준 운송비 가중치 배분 ⇒ 할당된 운송비(`freight_alloc`) 칼럼 생성
- 각 매입별 운송비 포함 매입단가(`purchase_cost`) 계산

▼ 병합된 `purchase_vendor` 데이터의 info

```
Int64Index: 2372474 entries, 0 to 2372473
Data columns (total 23 columns):
#   Column                                Dtype
---  -
0   inventory_id                          object
1   store_id                              object
2   brand_id                              object
3   item                                  object
4   size                                  object
5   volume                                float64
6   total_volume                          float64
7   order_volume                          float64
8   classification                        int64
9   vendor_id                             int64
10  vendor                                object
11  purchase_order_number                 int64
12  purchase_order_date                   datetime64[ns]
13  receiving_date                        datetime64[ns]
14  invoice_date                          object
15  pay_date                              object
16  dollars                               float64
17  quantity                              int64
18  purchase_price                        float64
19  purchase_cost                         float64
20  freight                               float64
21  freight_alloc                         float64
22  lead_time                             int64
dtypes: datetime64[ns](2), float64(8), int64(5), object(8)
```

▼ 2-3. 판매 데이터: `sales`

`sales` + `purchase_prices`

- 운송비 미포함 매입단가(`purchase_price`), 소매 판매가(`retail_price`) 칼럼 병합

▼ 병합된 `sales` 데이터 info

Int64Index: 12825363 entries, 0 to 12825362

Data columns (total 20 columns):

#	Column	Dtype
0	inventory_id	object
1	store_id	float64
2	brand_id	float64
3	item	object
4	size	object
5	sales_quantity	int64
6	sales_dollars	float64
7	sales_price	float64
8	sales_date	datetime64[ns]
9	item_volume	float64
10	classification	int64
11	excise_tax	float64
12	vendor_id	int64
13	vendor	object
14	month	int64
15	dayofweek	int64
16	city	object
17	volume	float64
18	purchase_price	float64
19	retail_price	float64

dtypes: datetime64[ns](1), float64(9), int64(5), object(5)

▼ 2-4. 선입선출 재고 추적 처리 : `sales_fifo`



`판매 (sales)` 데이터와 `매입 (purchases)` 데이터를 통합하여 보기 위해, **선입선출법(FIFO)**을 가정하고 데이터를 병합하였습니다.

▼ cf) 선입선출법(FIFO; First In, First Out)이란?

- 재고자산의 원가 계산 방법 중 하나로, 먼저 입고된 상품을 먼저 출고하는 방법입니다.

• 구현 방법

▼ 큐(Queue) 자료구조로 구현



Queue Data Structure

- 큐(Queue)란 **선입선출**의 자료구조로, 가장 앞에 들어온 값이 가장 먼저 제거되는 구조입니다.

1. 24/1/1의 `기초 재고 (inventory)`에 존재하는 인벤토리 고려하여 시작
2. `매입 (purchases)`에서 추가 매입 시, 큐에 추가
3. `판매 (sales)`에서 판매 시, 큐에서 제거



어떤 판매(sales)가 어떤 매입(purchase)에서 온 것인지 연결하여 병합하였습니다.



3. 매출총이익(gross profit), 이익률(gross margin rate) 계산

1. 매출총이익(gross profit)

- 순매출 = 433,088,711.28 = 452,062,952.02 - 18,974,240.74
 - sales 데이터의 sales_dollars 합계(총매출) - Excise_tax(특별 소비세)
- 총매출원가 = 3,147,414,08.57
 1. 기초재고에 있던 품목 : purchase_price * sales_quantity
 2. 당기매입 품목 : purchase_cost(운반비 포함) * sales_quantity
- 매출총이익 = 118,347,302.71
 - 순매출 - 총매출원가

2. 이익률(gross margin rate)

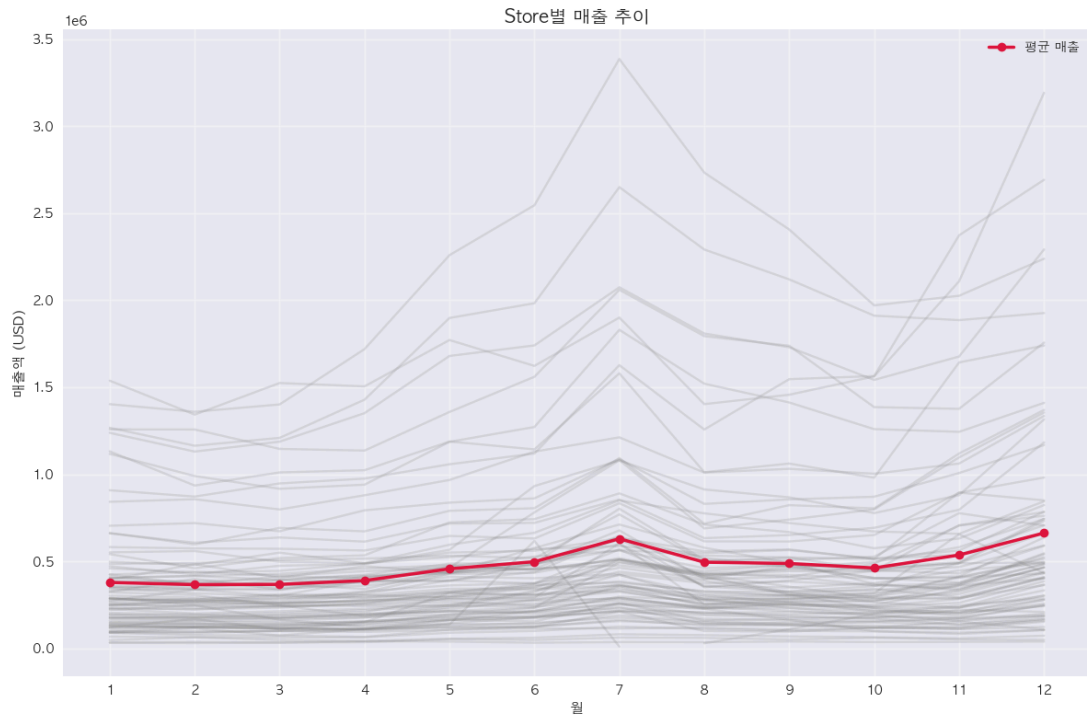
$$\text{매출총이익률} = \frac{\text{매출총이익}}{\text{순매출}} = 27.33\%$$



brand_id 군집화를 통한 특징 파악

1. brand_id를 기준으로 한 이유

▼ store_id 기준?



- store_id에 상관없이 매출 추이가 비슷하게 나타남
→ 7월, 12월의 매출이 크다는 것을 확인 가능.

▼ city 기준?

- 상권 특징을 알기 위해서는 **인구통계학적/지리적 데이터**가 필요합니다.
→ 우리가 갖고 있는 데이터만으로는 city 기준 분석을 통한 전략을 도출할 수 없다고 판단했습니다.



기초 대비 기말 재고가 증가한 것과 brand_id의 증가 사이에 큰 관련성이 있을 것이라는 앞선 추측에 더해, store_id나 city별로 나누어 분석함으로써 얻을 수 있는 인사이트가 적을 것이라고 판단함에 따라 brand_id를 기준으로 군집화를 진행하기로 결정했습니다.

2. brand_id 군집화

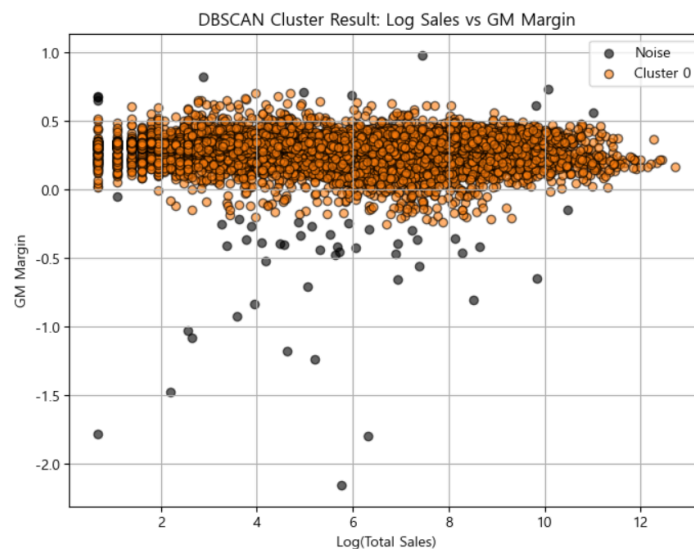
2-1. DBSCAN

- 변수: 판매량, 매출 총 이익률 (마진률)

- 결과

1. 비주류/특이 : 47개 → 매출 비중 낮고, 말 그대로 비주류/특이하므로 무시
2. 일반적 : 11190개 → 매출 비중 99.86% / 판매량, 매출 총 이익률 특이하지 않음

△ DBSCAN: 유효 군집이 2개 미만이라 실루엣 계산 불가
cluster_dbscan_log
0 11183
-1 54
Name: count, dtype: int64



	cluster_dbscan_log	avg_price_mean	month_std_mean	inv_std_mean	gm_margin_mean	total_sales_sum	total_revenue_sum	brand_count	sales_ratio(%)
1	0	34.908210	115.359450	76.239481	0.293723	32728772	4.556484e+08	11183	99.86
0	-1	15.502439	209.396628	77.132780	-0.367297	189104	6.513270e+05	54	0.14

2-2. K-means

- 변수: 판매가격, 월별 판매량 표준편차(계절성), 월별 재고량 표준편차
- 결과: (실루엣 계수 0.8630)

Aa cluster
<u>0. 일반적</u>
<u>2. 변동성 높음</u>
<u>1. 고가</u>

0. 일반적 : 10770개

- 매출 비중 약 60%
- 판매가격, 판매량 변동성, 재고량 변동성 평균적

1. 고가: 11개

- 매출 비중 약 0.04%
- 매우 고가, 판매량 변동성, 재고량 변동성 매우 낮음

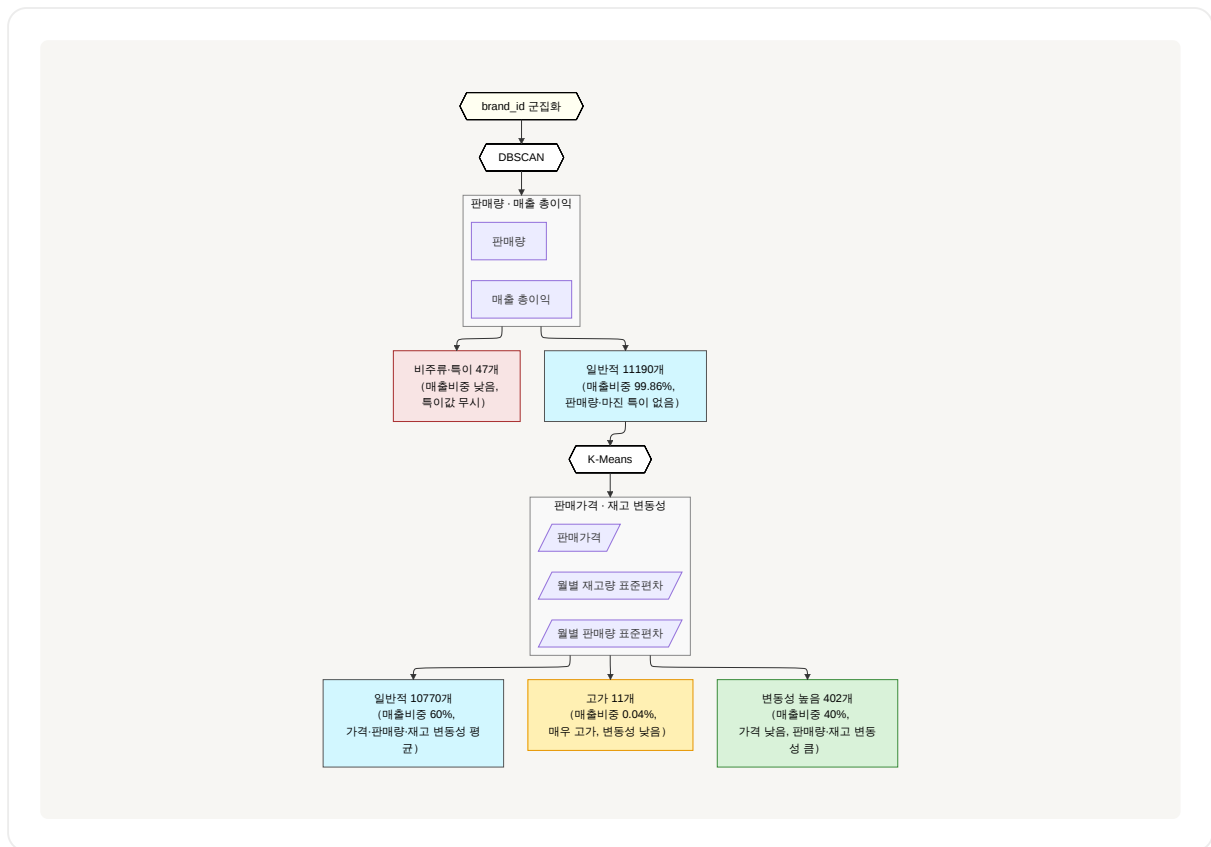
2. 변동성 높음: 402개

- 매출 비중 약 40%
- 가격 낮음, 판매량 변동성, 재고량 변동성 매우 큼

kmeans_cluster	avg_price_mean	month_std_mean	inv_std_mean	count
0	0	32.236652	64.915141	42.470162
1	1	3458.050606	0.568787	0.793920
2	2	12.813964	1469.956227	983.019237

kmeans_cluster	cluster_total_dollars	sales_ratio
0	0	2.708653e+08
1	1	1.640541e+05
2	2	1.846191e+08

2-3. 군집화 구조 정리



3. brand_id별 군집화 결과 해석

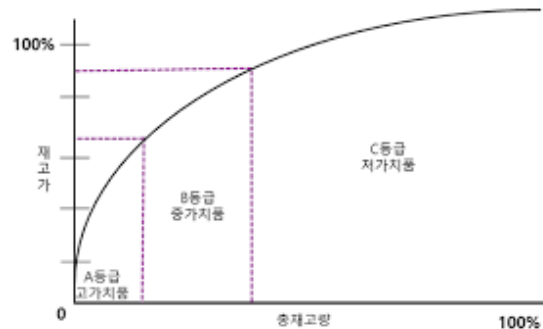
3-1. [군집 0] 일반적인 10770개 상품 관리 전략 → ABC 관리법



ABC 관리법(ABC inventory planning)

품목	내용	관리정도	주문주기	안전재고
A	가치(매출총이익)는 크지만 판매량이 적은 품목	정밀관리	짧음	소량
B	가치와 판매량이 중간에 속하는 품목	정상관리	중간	중량
C	가치는 적지만 판매량은 많은 품목	대강관리	깊	대량

- 소수가 큰 중요도를 차지하고 다수가 낮은 중요도를 차지한다는 파레토 법칙을 이용



- 기업이 수천 개의 재고를 모두 정확히 관리하는 것은 상당히 어려운 일
- 따라서 **재고의 가치와 중요도에** 따라 품목을 A등급, B등급, C등급 등으로 분류하여 구분 관리하는 기법이 필요함

▼ A등급 / B등급 / C등급 범주화 전략

1. 군집 0의 brand_id 리스트를 추출해, sales 데이터셋을 해당 brand_id 별로 필터링
2. brand_id 별 총 재고 가치 = 매출총이익 (gross_profit) * 판매량 (sales_quantity)
3. 이를 내림차순으로 정렬 후, 누적 비율 (cumulative ratio) 도출
 - A그룹: 상위 누적 60%
 - B그룹: A그룹을 제외한 상위 누적 25%
 - C그룹: 하위 누적 15%

⇒ 각 등급이 판매량에서 차지하는 비율과 판매가치 점유율이 반대 순서라는 것을 확인 가능

각 ABC 그룹별 총 가치 점유율:

ABC_class

A 0.599785

B 0.250125

C 0.150091

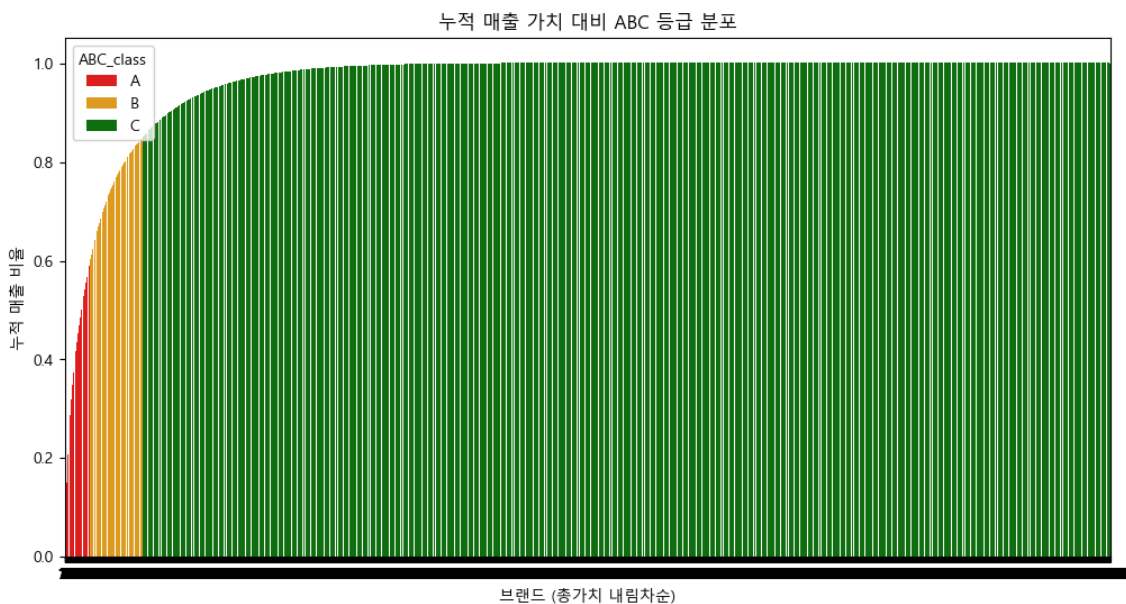
ABC 비율 (주문 단위 기준):

ABC_class

C 0.491622

B 0.274408

A 0.233970



3-2. [군집 1] 초고가 제품 11개 상품 관리 전략 → 선주문 후발주 방식



선주문 후발주 방식

- 초고가이므로 정밀관리가 필요해 재고 관리비용이 많이 소요 따라서 최대한 재고를 적게 보유하는 전략이 필요
- 1번 군집에 대해서는 선주문 후발주 방식으로 관리한다면 재고 관리비용을 줄일 수 있을 것

3-3. [군집 2] 변동성 높은 402개 상품 → 회귀분석

- 402개의 제품이 전체 매출의 40%를 차지함! 이는 소수의 핵심 제품이 매출 구조를 주도하고 있음을 보여줌



군집 2 회귀 분석 및 기대효과

1. 회귀분석 목적

- **목적:** 변동성이 큰 제품군에 대해 일별 제품 수요량을 예측하여 효율적인 재고 관리 및 발주 계획을 수립하기 위함
- **분석 단위:** brand_id별 일별 판매 데이터를 기준으로 회귀모델 구축

2. 회귀분석 모델 설명

결과 요약

- R-squared : 0.671
- 회귀 계수
 - 전일 판매량 ($\beta = +2.3$)
 - 전일 많이 팔릴수록 당일 수요도 높음.
 - 전일 재고량 ($\beta = +1.2$)
 - 전일 재고가 충분할수록 다음날 판매가 늘어남.
 - 판매가격 ($\beta = -0.07$)
 - 가격이 높을수록 수요가 약간 감소하나, 영향은 제한적.
 - 평균 보유기간 ($\beta = -0.50$)
 - 평균 재고 보유기간이 길수록 수요 감소 경향.
 - 평균 리드타임 ($\beta = -0.02$)
 - 리드타임이 길어질수록 수요 충족률 하락.

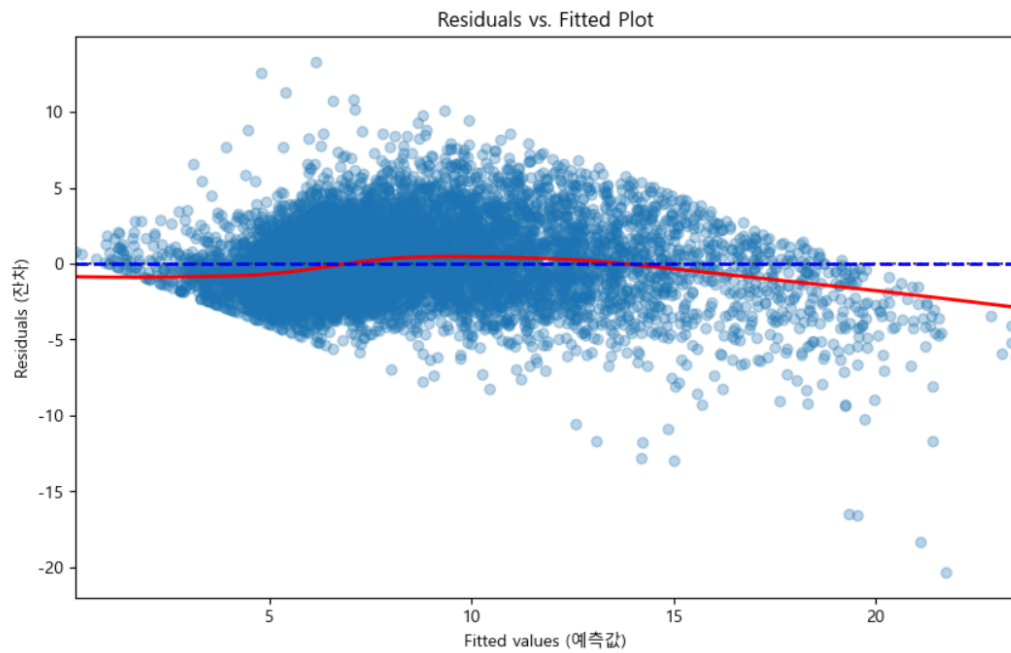
결과 해석

- 전일 판매 데이터와 재고 데이터가 다음날 수요 예측의 핵심 변수!
- 재고 부족 시 기회 손실 발생 가능 → **적정 재고 확보**가 **수요 충족**에 직접적 기여.
- 가격보다는 **재고 및 운영 효율**이 수요에 더 큰 영향.
- 재고 과잉은 회전율을 낮추고 수요 예측 정확도도 저하시킴 → 재고 최적화 필요.
- 입고 지연 시점에서 판매 기회 손실 가능 → 리드타임 단축이 중요.

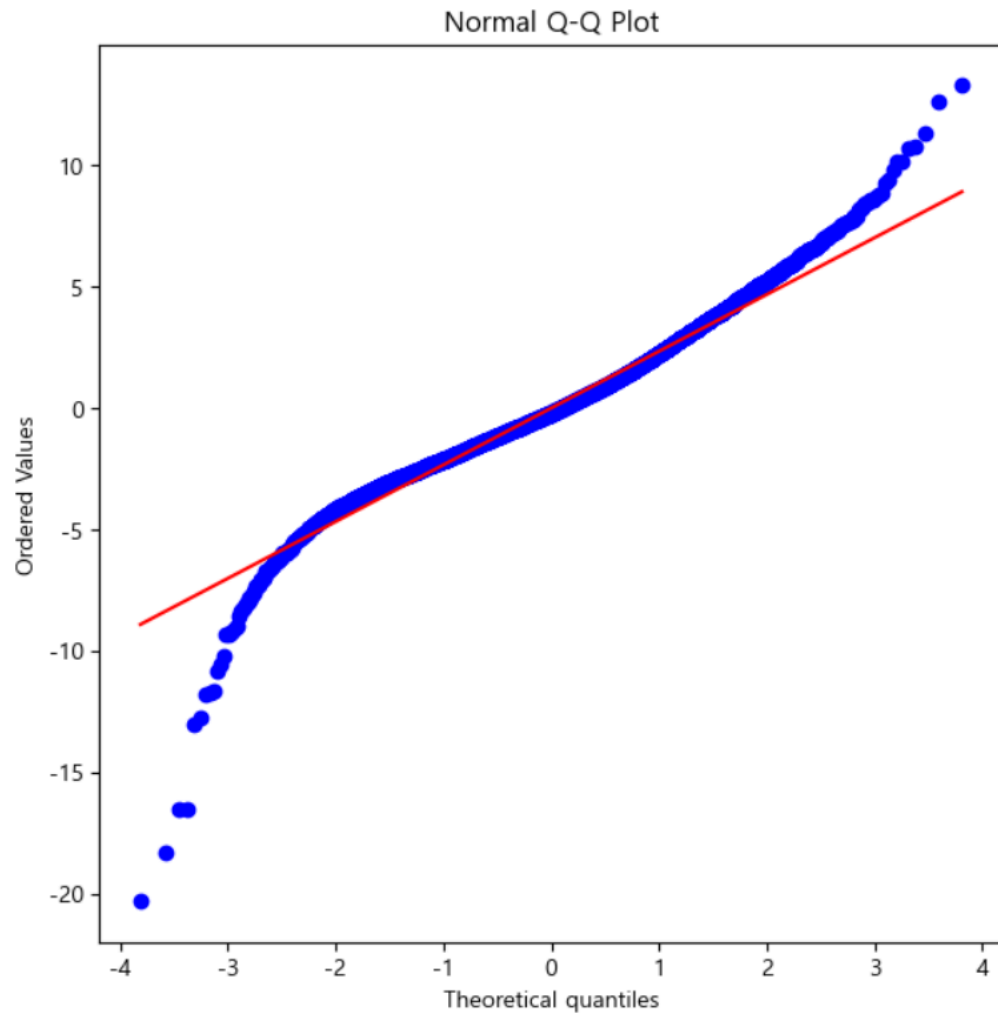
▼ cf) 회귀 조건 충족 확인

▼ 선형성, 등분산성

--- 1 & 3. 선형성/등분산성---



▼ 오차 정규성



▼ 독립성

--- 4. 오차의 독립성 (Durbin-Watson) ---
Durbin-Watson Statistic: 1.510

▼ 다중공선성

```
--- 5. 다중공선성 (VIF) ---
      feature      VIF
0    prev_day_sales 1.561416
1  prev_day_inventory 1.612901
2          avg_price 1.132140
3    category_Gin   1.049732
4  category_Liqueur 1.014375
5    category_Rum   1.007509
6  category_Tequila 1.018360
7    category_Vodka 1.035193
8  category_Whiskey 1.054313
9    category_Wine  1.059998
```

4. 회귀 분석 결과에 따른 군집 2 전략 제시

4-1. 군집 2의 특징

- 매출 비중(40%)은 높지만 브랜드 수(402개)는 적은 **핵심 군집**
- 판매량·재고량 **변동성이 높음**

재고 부족 문제

- 402개의 **brand_id** 모두 재고 수량이 0이 된 적이 없음
- 402개의 **brand_id** 모두 일별 예상 수요량보다 재고수량이 부족했던 적도 없음

재고 과잉 문제

- 반면, 일별 예상 수요량보다 재고 수량이 많아 과잉 재고가 존재한 경우는 매우 많음
- 그러므로 모든 지점들은 판매량 변동성이 높은 402개의 상품들을 재고를 많이 쌓아 두는 방식으로 관리하고 있음

	brand_id	날짜	예상_수요량	실제_판매량	가용_재고량	과잉_수량	부족_수량	과잉비용	부족비용(매출상실)
0	31915	2024-05-01	25.688495	2	2213.0	2187.311505	0.0	18089.066147	0.0
1	31915	2024-05-02	23.032274	8	2222.0	2198.967726	0.0	18185.463094	0.0
2	31915	2024-05-03	24.795345	19	2214.0	2189.204655	0.0	18104.722500	0.0
3	31915	2024-05-04	28.065601	19	2251.0	2222.934399	0.0	18383.667484	0.0
4	31915	2024-05-05	28.263399	23	2232.0	2203.736601	0.0	18224.901693	0.0



재고 부족보다는 재고 과잉 문제가 더 해결해야 할 문제이다!
적절한 발주 전략을 통해 **재고 과잉으로 인한 손실을 절감**할 수 있다!

4-2. 전략 기대효과 검증

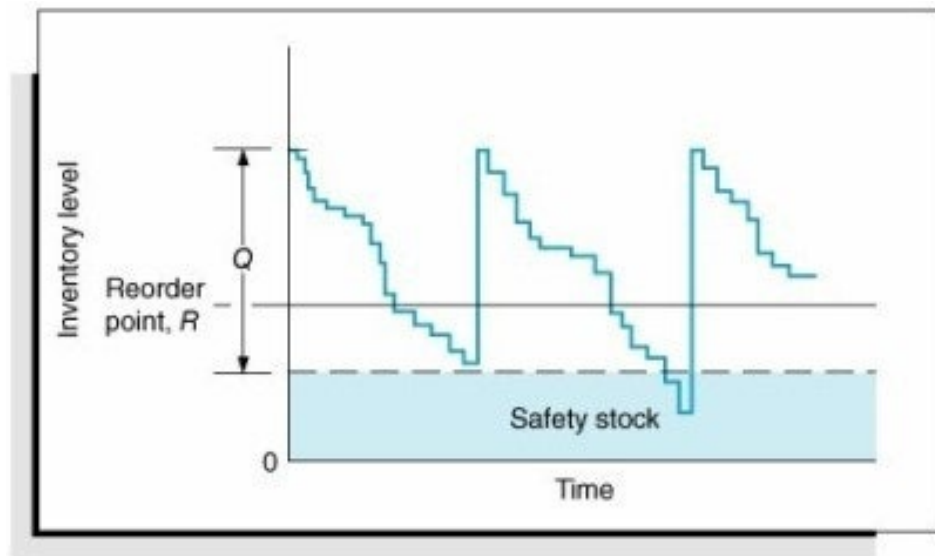
| 기대효과 검증 방법

- 변동성이 높은 기간을 찾음
 - 6월, 7월, 11월, 12월
- 해당 기간에 판매량 변동성이 가장 높은 `store_id` 선정
 - `store_id == 61`
- 군집 2의 모든 `brand_id`

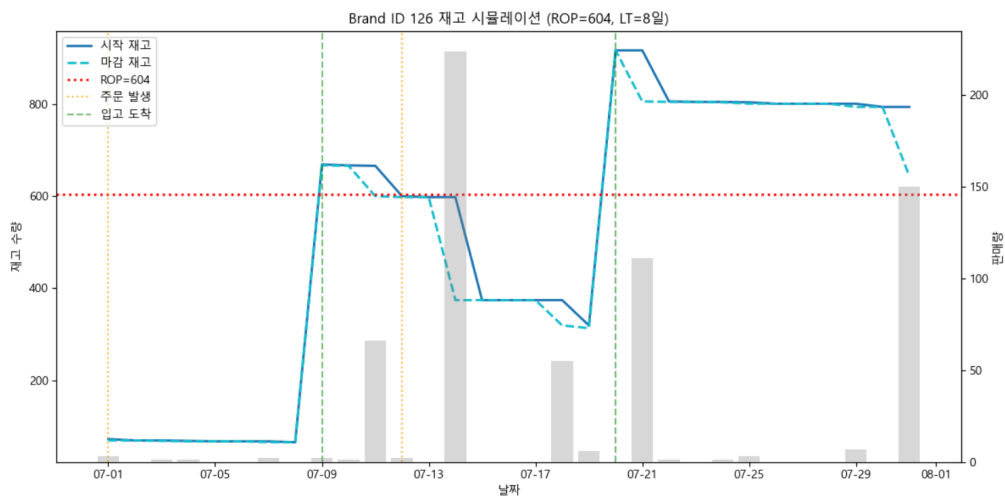
| 기대효과 계산 방법

- 재고 수량이 **재주문점 밑으로 떨어지는 즉시 필요한 만큼만 발주**를 하는 전략

- **재주문점(ROP)** = 리드타임 동안의 예상 수요량 + 안전재고
- **안전재고** = $Z \times \sigma_D \times \sqrt{L}$
 - Z (Z-값): 원하는 서비스 수준에 따라 결정되는 값
 - ex) 95% 서비스 수준을 목표로 한다면, Z값은 약 1.645이다.
 - σ_D (수요의 표준편차): 특정 기간 동안의 판매량 데이터에서 계산된 표준편차
 - L (리드타임): 공급업체로부터 주문한 상품이 도착하기까지 걸리는 평균 시간
- **주문량** = 리드타임 동안의 예상 수요량 + 안전재고



재주문점 이론



재주문점 적용 시뮬레이션 결과

4-3. 기대효과 시뮬레이션 결과

- 실제 데이터의 `store_ID == 61`의 7월에 적용
: 실제 총 과잉 재고 수량 vs 시뮬레이션 총 과잉 재고 수량 비교

▼ 결과 비교

```
=====
--- [ Store ID 61, 7월 총 과잉 재고 '수량' 비교 ] ---
(전략: Z=1.28, SS 상한 50% 적용)

1. 실제 총 과잉 재고 수량: 549,283 개
2. 시뮬레이션 총 과잉 재고 수량: 269,431 개

-----
>> ROP/주문량 전략 적용 시 '수량' 절감: 279,852 개 (50.95 % 감소)
=====
```

```
=====
--- [ Store ID 50, 7월 총 과잉 재고 '수량' 비교 ] ---
(전략: Z=1.28, SS 상한 50% 적용)

1. 실제 총 과잉 재고 수량: 910,053 개
2. 시뮬레이션 총 과잉 재고 수량: 473,770 개

-----
>> ROP/주문량 전략 적용 시 '수량' 절감: 436,283 개 (47.94 % 감소)
=====
```



ROP 기반 맞춤 발주 전략 적용시 과잉 재고 수량의 약 50% 절감 가능!

 **결론: brand_id별 재고 관리 가이드라인**

0 0번 군집에 해당하는 일반적인 10777개 상품 관리 전략

- ABC 관리 전략: 소수의 고가 품목(A)이 전체 재고 가치의 대부분을 차지하고, 다수의 저가 품목(C)은 상대적으로 낮은 비중을 차지함
→ **품목별 관리 수준**을 효율적으로 차별화

1 1번 군집에 해당하는 초고가 11개 상품 관리 전략

- 선주문-후발주 방식 재고 관리

2 2번 군집에 해당하는 변동성 높은 402개 상품 전략

- 재주문점(ROP) 기반 맞춤 수량 발주 전략: 재고 수준이 '**재주문점(ROP)**'에 도달했을 때, '미리 정해진 양(Q)'만큼 주문하는 방식 (이 경우 $Q = LTD + SS$)